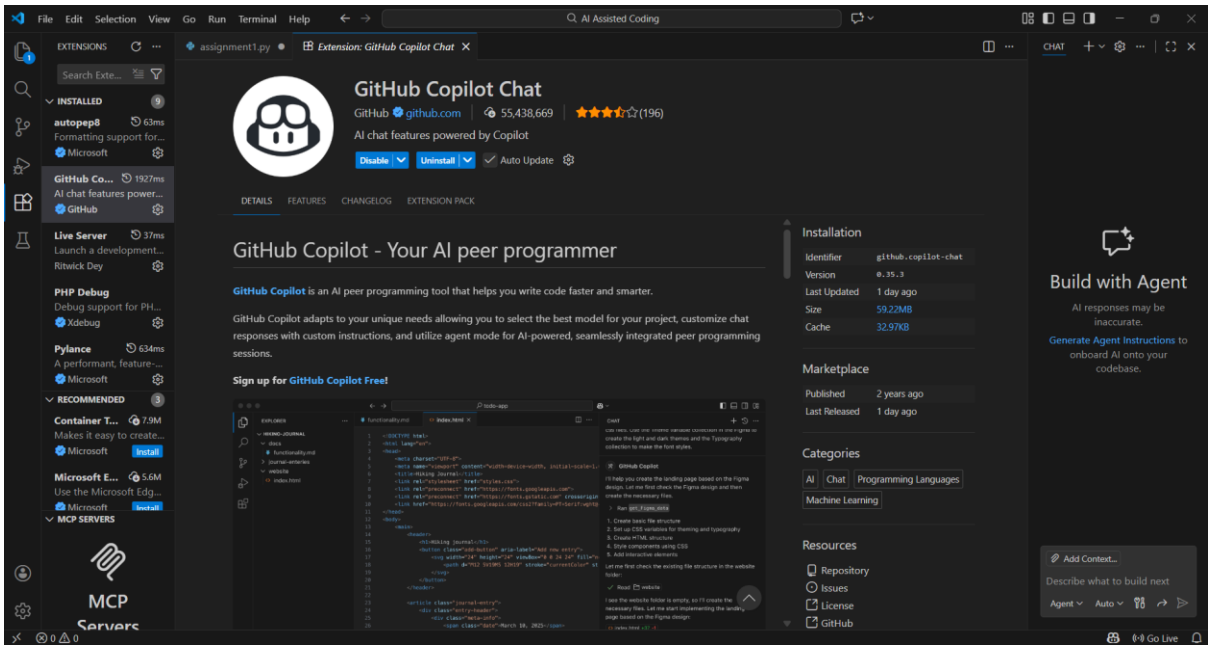
# ASSIGNMENT – 1

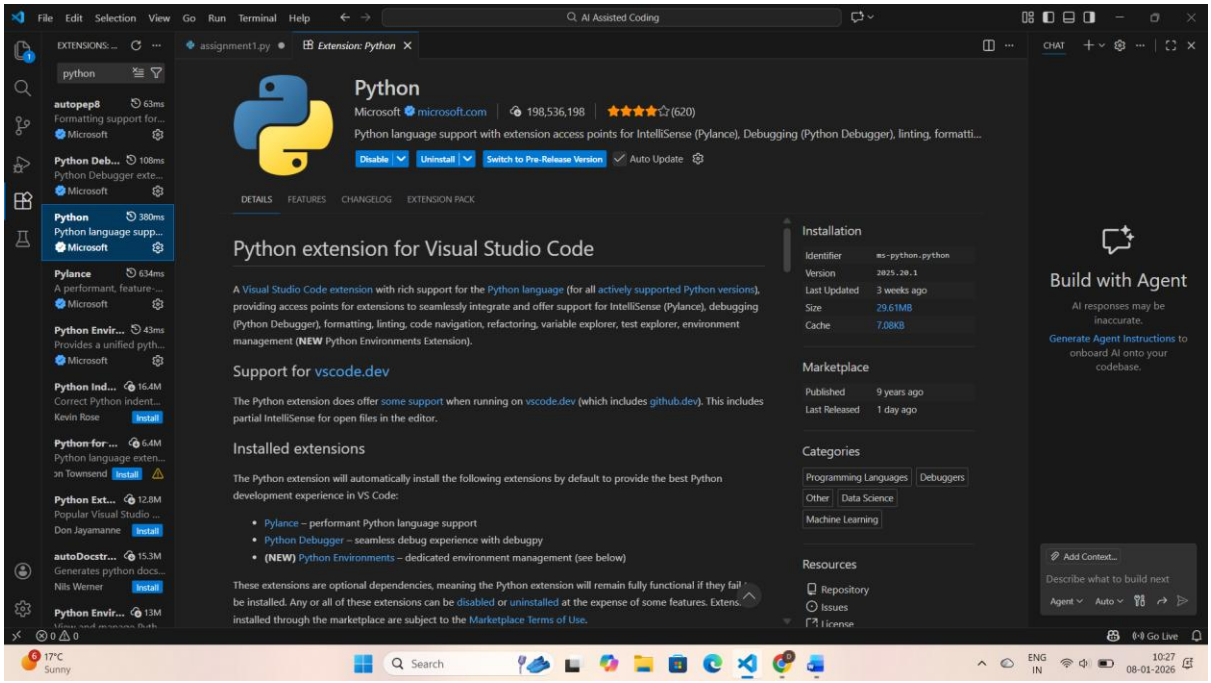
D.Charmi

2303A51314

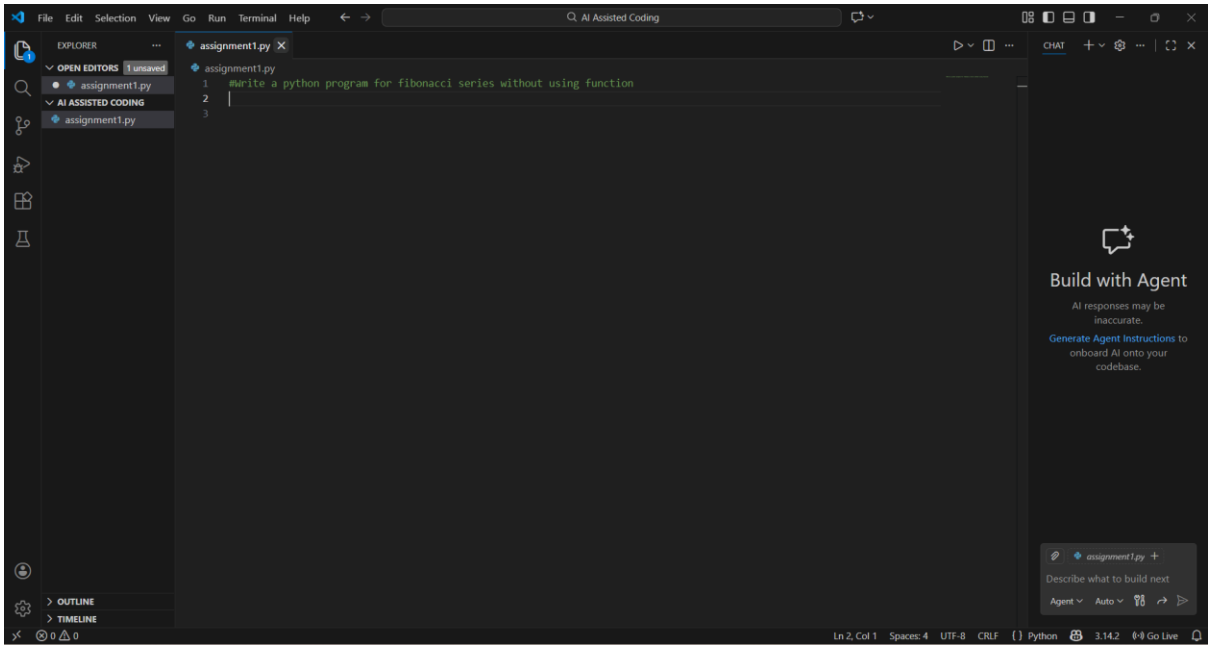
Batch - 05

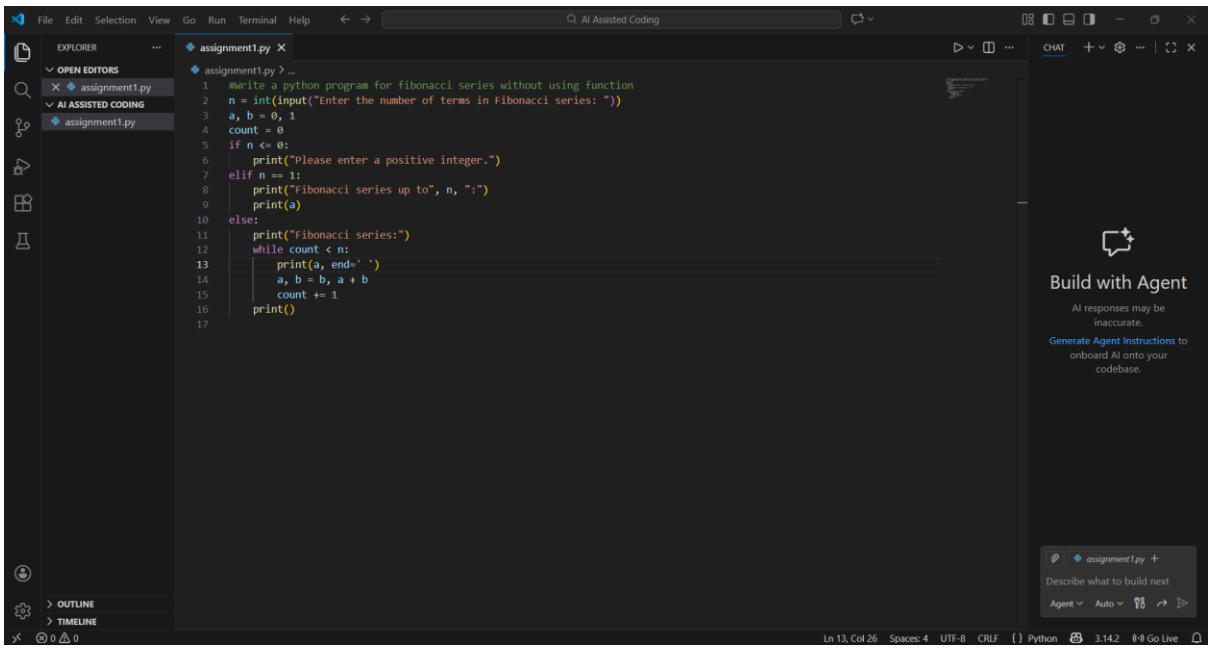
## Task – 0 : Installation of GitHub Copilot



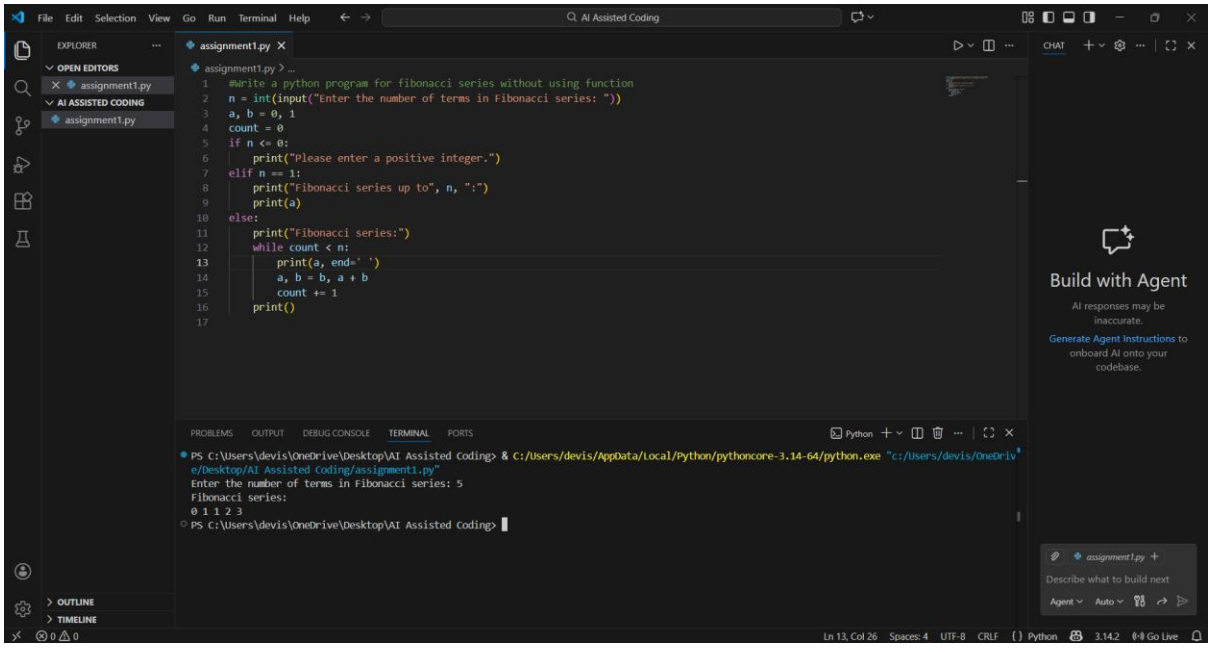



**Task-1:** Prompt


File Edit Selection View Go Run Terminal Help
AI Assisted Coding
EXPLORER
OPEN EDITORS 1 unsaved
assignment1.py
AI ASSISTED CODING
assignment1.py
assignment1.py
#Write a python program for fibonacci series without using function
CHAT
Build with Agent
AI responses may be inaccurate.
Generate Agent Instructions to onboard AI onto your codebase.
Describe what to build next
Agent
Auto
OUTLINE
TIMELINE
Ln 2, Col 1
Spaces: 4
UTF-8
CRLF
Python
3.14.2
Go Live


File Edit Selection View Go Run Terminal Help
AI Assisted Coding
EXPLORER
OPEN EDITORS
assignment1.py
AI ASSISTED CODING
assignment1.py
assignment1.py
#Write a python program for fibonacci series without using function
n = int(input("Enter the number of terms in Fibonacci series: "))
a, b = 0, 1
count = 0
if n <= 0:
print("Please enter a positive integer.")
elif n == 1:
print("Fibonacci series up to", n, ":")
print(a)
else:
print("Fibonacci series:")
while count < n:
print(a, end=' ')
a, b = b, a + b
count += 1
print()
CHAT
Build with Agent
AI responses may be inaccurate.
Generate Agent Instructions to onboard AI onto your codebase.
Describe what to build next
Agent
Auto
OUTLINE
TIMELINE
Ln 13, Col 26
Spaces: 4
UTF-8
CRLF
Python
3.14.2
Go Live

# Output:

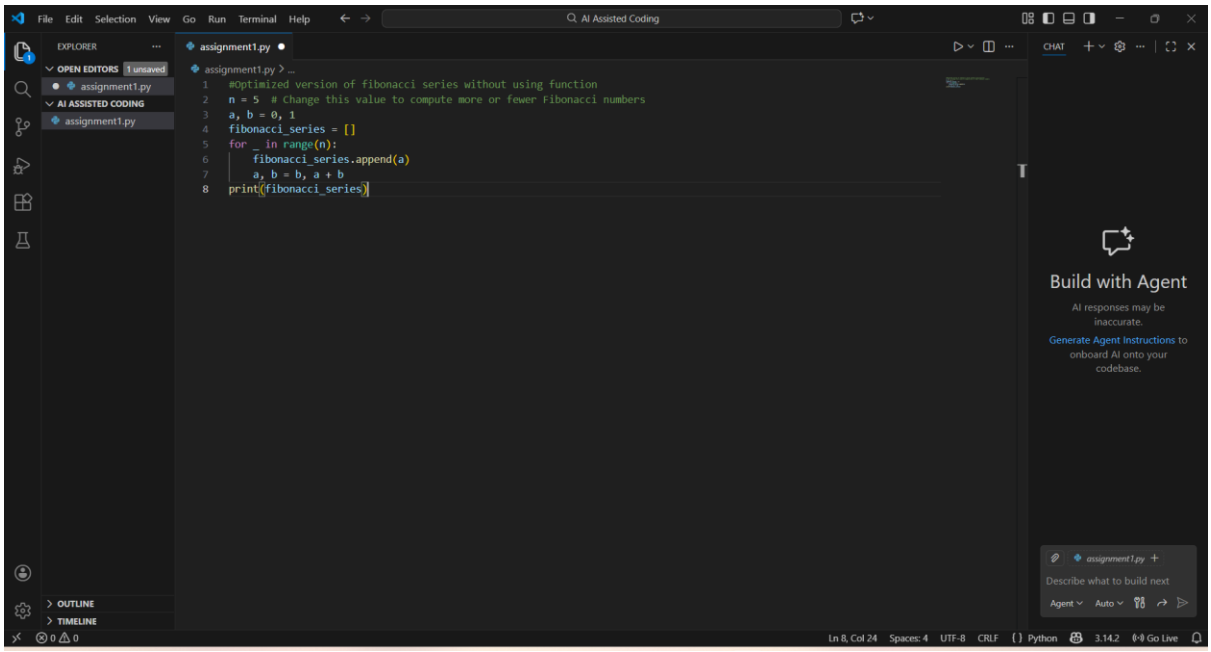



## Explanation :

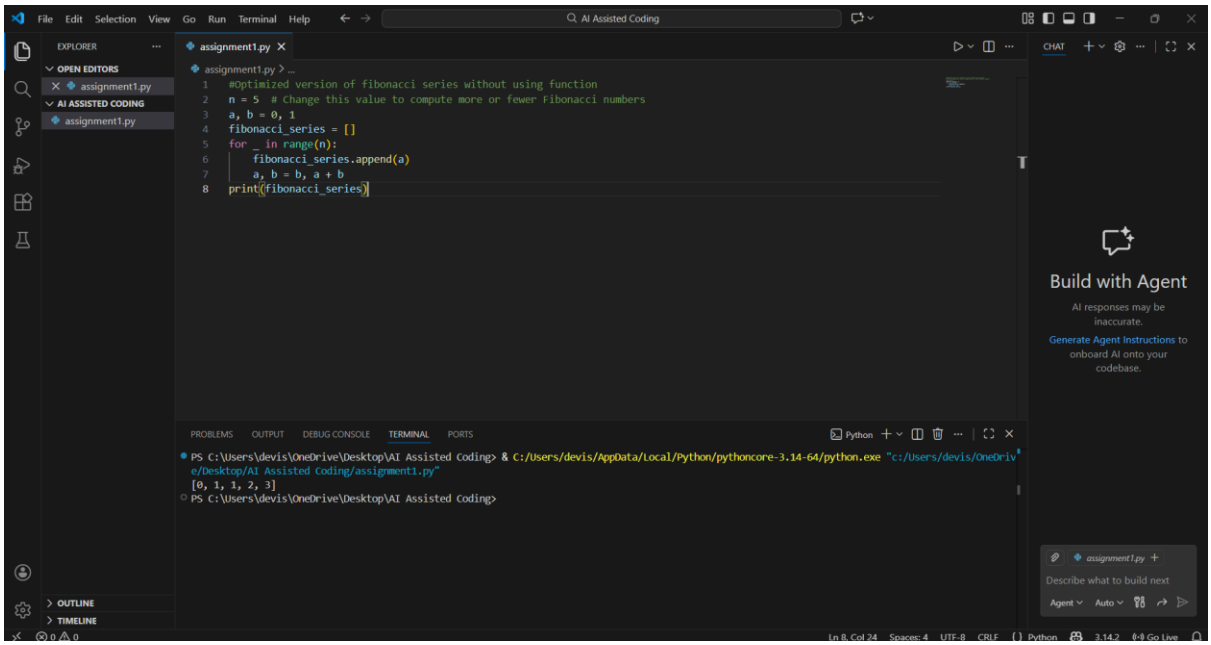
This program generates the Fibonacci series up to 'n' terms without using any functions. It first takes an integer input 'n' from the user, which represents the number of terms to be printed. It initializes two variables 'a' and 'b' to 0 and 1, which are the first two terms of the Fibonacci series. A counter variable 'count' is also initialized to keep track of the number of terms printed. The program checks if 'n' is less than or equal to 0, in which case it prompts the user to enter a positive integer. If 'n' is 1, it prints only the first term of the series (0). For values of 'n' greater than 1, it enters a while loop that continues until 'count' is less than 'n'. Inside the loop, it prints the current value of 'a', then updates 'a' and 'b' to the next two terms in the series. The values of 'a' and 'b' are updated using tuple unpacking: 'a' takes the value of 'b', and 'b' takes the sum of the previous 'a' and 'b'. The counter 'count' is incremented by 1 in each iteration. Finally, it prints the complete Fibonacci series up to 'n' terms.

# Task-2

**Prompt :** #Optimized version of fibonacci series without using function




# Output :

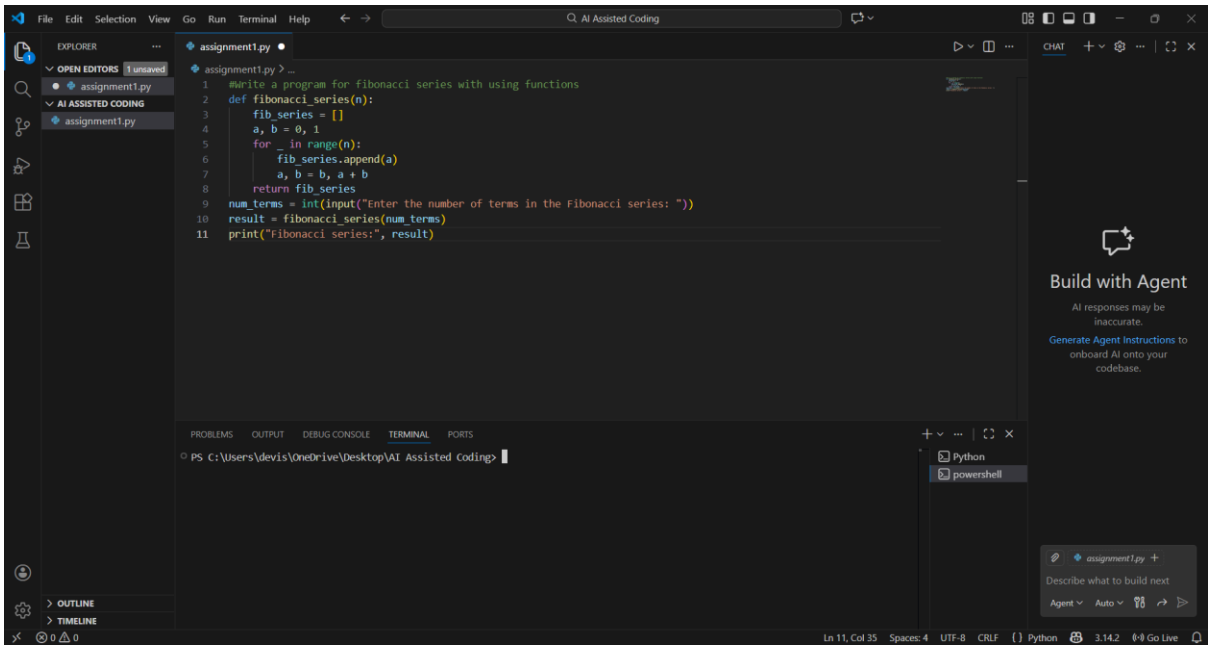


**Explanation** : # This code generates the first 'n' numbers in the Fibonacci series using an iterative approach.

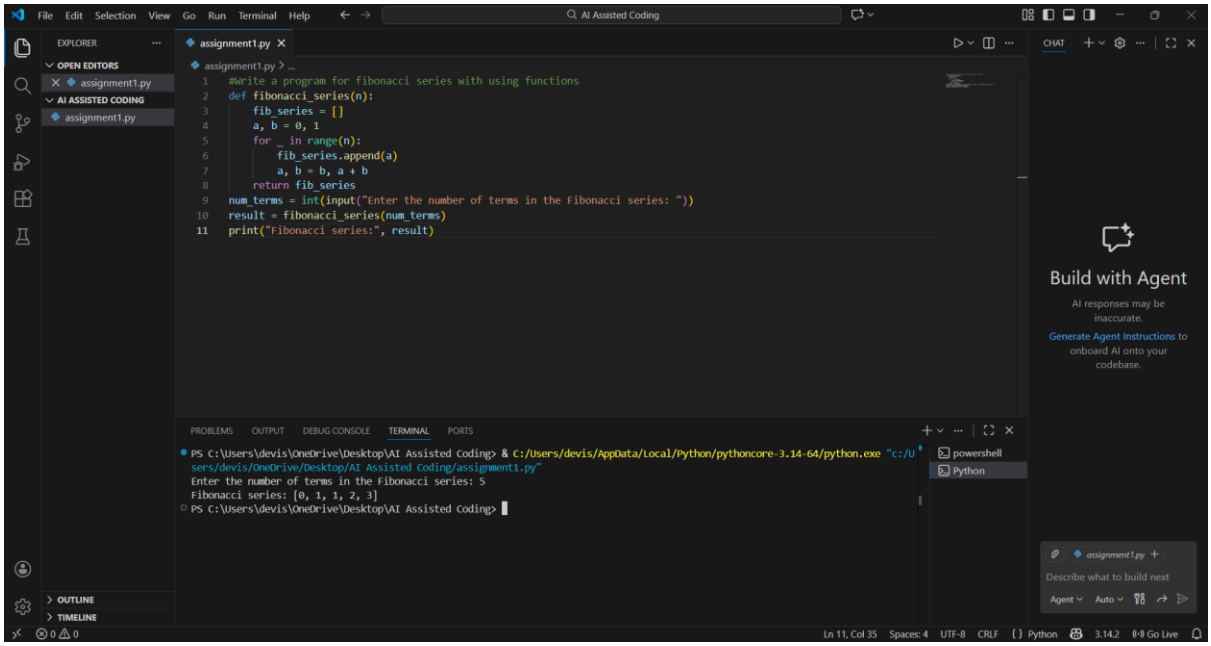
It initializes two variables 'a' and 'b' to represent the first two Fibonacci numbers (0 and 1). In each iteration of the loop, it appends the current value of 'a' to the list and then updates 'a' and 'b' to the next two numbers in the series. This method is efficient in terms of both time and space complexity.

## Task – 3:

**Prompt :** #Write a program for fibonacci series with using functions




**Output :**

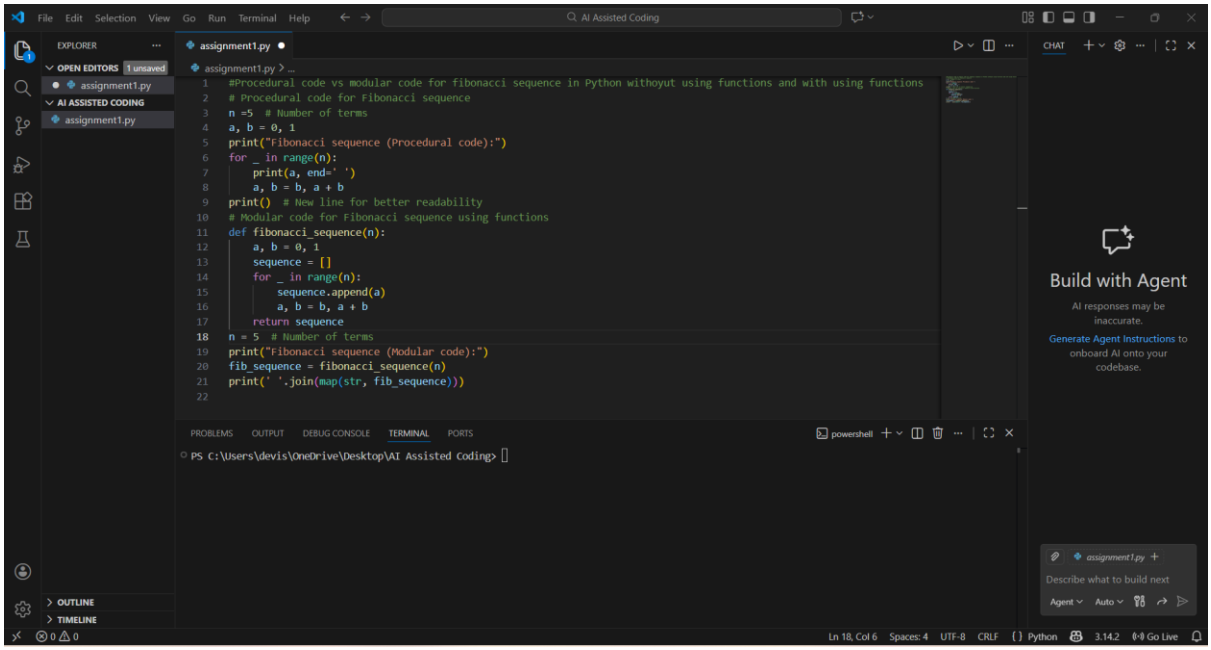



## Explanation :

This program defines a function called fibonacci_series that takes an integer n as input and returns a list containing the first n terms of the Fibonacci series. Inside the function, we initialize an empty list fib_series to store the Fibonacci numbers. We also initialize two variables a and b to represent the first two numbers in the series (0 and 1). We then use a for loop to iterate n times. In each iteration, we append the current value of a to the fib_series list, and then update a and b to the next two numbers in the series using tuple unpacking. After defining the function, we prompt the user to input the number of terms they want in the Fibonacci series. We convert this input to an integer and store it in the variable num_terms. We then call the fibonacci_series function with num_terms as the argument and store the result in the variable result. Finally, we print the resulting Fibonacci series

## Task – 4 :

**Prompt :** #Procedural code vs modular code for fibonacci sequence in Python without using functions and with using functions




# Output :

**Explanation :**

1. Code Clarity:

- Procedural Code: The logic is straightforward but can become cluttered as the program grows.

- Modular Code: The use of functions makes the code more organized and easier to read. Each function has a specific purpose.

2. Reusability:

- Procedural Code: The code is not reusable. If you need to generate the Fibonacci sequence in another part of the program, you would have to duplicate the code.

- Modular Code: The function can be reused anywhere in the program or in other programs without rewriting the logic.

3. Debugging:

- Procedural Code: Debugging can be more challenging as the entire logic is in one place. Identifying issues may require going through the whole code.

- Modular Code: Debugging is easier since you can isolate issues within specific functions. You can test functions independently.

4. Maintainability:

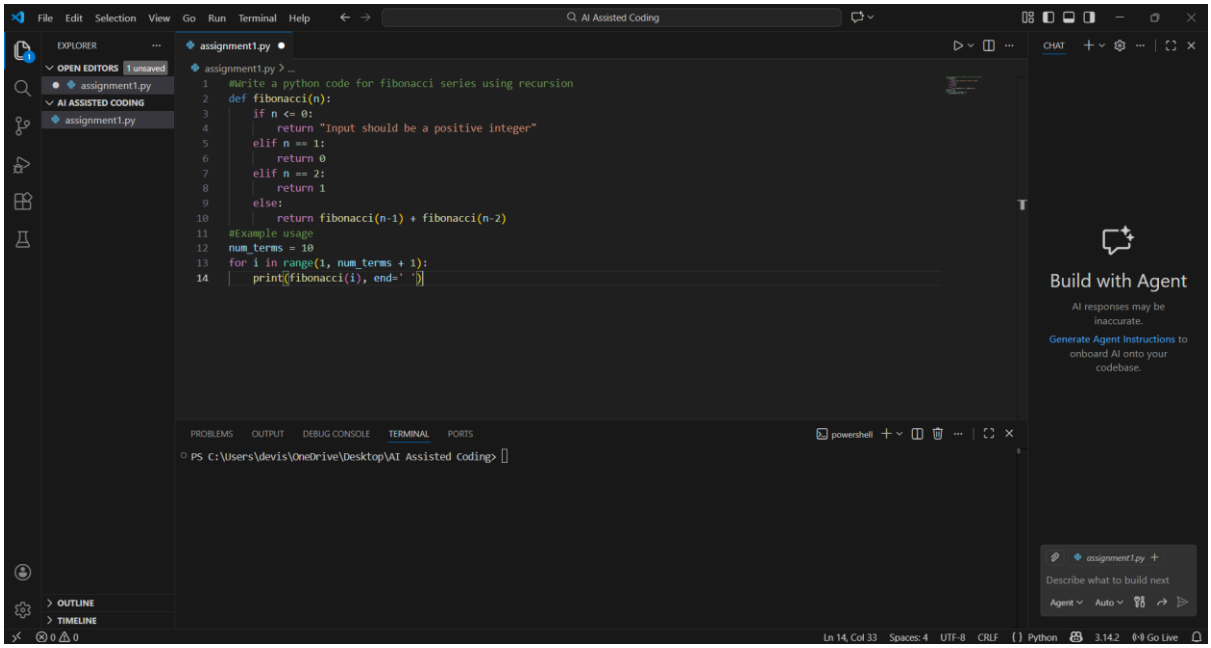
- Procedural Code: Maintaining the code can be difficult as changes may require modifications in multiple places.

- Modular Code: The code is easier to maintain. Changes can be made within functions without affecting the overall structure of the program.

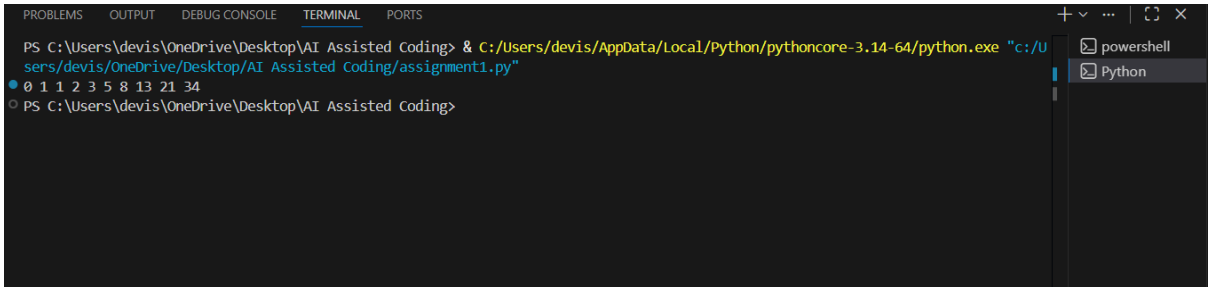
Overall, modular code is generally preferred for larger and more complex programs due to its advantages in clarity, reusability, debugging, and maintainability.

## Task – 5:

**Prompt :** #Write a python code for fibonacci series using recursion




## Output :




## Explanation :

The fibonacci function takes a positive integer n as input and returns the nth term in the Fibonacci series. The base cases are defined for n = 1 and n = 2,

returning 0 and 1 respectively. For any n greater than 2, the function calls itself recursively to calculate the sum of the two preceding terms in the series (fibonacci(n-1) + fibonacci(n-2)).The example usage demonstrates how to print the first 10 terms of the Fibonacci series by calling the fibonacci function in a loop.